

ANGULARJS HANDS-ON ACTIVITY

TechFest Event Registration Portal

Target Audience: Second Year MCA Students

Difficulty Level: Intermediate

Estimated Duration: 90 Minutes

Theme: Corporate Bootcamp

CORPORATE SCENARIO

Welcome to the Corporate Bootcamp. Your software development agency has been hired by a leading university to develop a modern **TechFest Event Registration Portal**.

The application should allow organizers to seamlessly manage events and register participants using a dynamic, single-page application built entirely with AngularJS.

Development Constraints

As a developer on this project, you must strictly adhere to the following architectural guidelines:

- Complete the project using **ONLY** the fundamental AngularJS concepts covered in the recent training modules.
- **Do NOT** implement Routing mechanisms.
- **Do NOT** use custom Services or Factory providers.
- **Do NOT** make HTTP calls to external APIs.

LEARNING OBJECTIVES

By successfully completing this bootcamp activity, you will demonstrate proficiency in the following core AngularJS concepts:

- Initialize and bootstrap an AngularJS application.
- Define and configure Controllers to manage application logic.
- Utilize `$scope` variables to bridge the View and Controller.
- Implement Two-way Data Binding for real-time UI updates.
- Work effectively with built-in AngularJS Directives.
- Validate HTML forms using AngularJS state properties.
- Handle form submissions efficiently using `ng-submit`.
- Render dynamic lists and tabular data using `ng-repeat`.

TASK 1: CORE DASHBOARD SETUP

Requirement: Create a primary dashboard for the application displaying key event metrics. The dashboard must show:

- Event Name
- Organizer Name
- Total Participants
- Event Date



TECHNICAL HINT

Use the `ng-model` directive to bind input fields to your scope variables, allowing organizers to edit the dashboard values dynamically in real-time.

THINK BEFORE YOU CODE

- How does changing a model in an input element instantly reflect in a standard expression (`{{ }}`)?
- What is the difference between one-way binding and two-way binding in this context?

DEVELOPER NOTES / VARIABLE MAPPING

```
// Define initial states in controller  
  
SCOPE VARIABLE 'eventName'      = "TechFest 2026"  
  
SCOPE VARIABLE 'organizer'      = "University Administration"  
  
SCOPE VARIABLE 'participantCount' = 0  
  
SCOPE VARIABLE 'eventDate'      = Current Date Object  
  
  
  
// UI Strategy  
  
Bind 'eventName' to an input using ng-model.  
  
Display 'eventName' in a header using {{ }} expression.
```

TASK 2: PARTICIPANT LIST INITIALIZATION

Requirement: Ensure that the portal can display an existing, hardcoded list of pre-registered participants. This serves as the foundation for the dynamic data table you will build later.



TECHNICAL HINT

Define an array of objects in your controller. Use the `ng-repeat` directive on an HTML element (like a list item or table row) to iterate over this array.



THINK BEFORE YOU CODE

- What happens to the UI if the participant array is completely empty upon initialization?
- How does AngularJS track individual elements within an array when updating the DOM?

DEVELOPER NOTES / ARRAY STRUCTURE PLANNING

```
// Initializing the Data Source  
  
SCOPE ARRAY 'participants' = [  
  OBJECT { Name: "Alice", Email: "alice@student.edu" },  
  OBJECT { Name: "Bob",    Email: "bob@student.edu" }  
]  
  
  
// UI Strategy  
  
Target element: <tr>  
  
Directive: ng-repeat iterating over 'participants' array
```

TASK 3: DYNAMIC CATEGORY SELECTION

Requirement: Create a dropdown menu that allows users to select an Event Category. The available categories must be dynamically rendered from an array.

Categories to include:

- AI (Artificial Intelligence)
- Web Development
- Cyber Security
- Cloud Computing
- Robotics
- IoT (Internet of Things)



TECHNICAL HINT

While `ng-repeat` can be used on `<option>` tags, the `ng-options` directive is heavily optimized for generating `<select>` elements and binding to complex objects.



THINK BEFORE YOU CODE

- Why is `ng-options` preferred over `ng-repeat` for select dropdowns in corporate applications?
- What value gets assigned to the model when a user selects an option?

DEVELOPER NOTES / DROPDOWN LOGIC

```
// Configuring the Dropdown  
  
SCOPE ARRAY 'categories' = [ 'AI', 'Cyber Security',  
    'Robotics', ... ]
```

SCOPE VARIABLE 'selectedCategory' = Default to first item
// UI Strategy
Target element: <select>
Directive: ng-options mapping individual category in 'categories'
array

TASK 4: CONDITIONAL INFORMATION RENDERING

Requirement: Below the dropdown created in Task 3, display specific details or warning messages based on the selected category.

For example, selecting "Cyber Security" might display "Note: Bring your own Kali Linux environment."



TECHNICAL HINT

Use the `ng-switch`, `ng-switch-when`, and `ng-switch-default` directives to conditionally swap DOM structures based on the dropdown's model value.



THINK BEFORE YOU CODE

- How does `ng-switch` differ from `ng-show` or `ng-hide` in terms of browser DOM manipulation?

- Which approach is better for application performance if the hidden elements contain heavy bindings?

DEVELOPER NOTES / CONDITIONAL MAPPING

// Mapping selections to UI elements
IF selection IS "AI"
-> Render Element: "AI requires a Google Colab setup."
IF selection IS "Cyber Security"
-> Render Element: "Bring your own Kali Linux environment."
IF selection IS "Web Development"
-> Render Element: "Pre-install Node.js and VS Code."
DEFAULT
-> Render Element: "Standard track guidelines apply."

TASK 5: DATA FORMATTING WITH FILTERS

Requirement: Ensure all data presented on the dashboard is formatted to corporate standards. Apply filters directly in the view to transform the output.

Implement the following filters across your dashboard and participant list:

- uppercase (e.g., for Event Codes)
- lowercase (e.g., for email addresses)
- currency (e.g., for registration fees, if applicable)
- number (e.g., to format large participant counts)
- date (e.g., to format the event start date)



TECHNICAL HINT

Remember that AngularJS filters use the pipe (|) operator within an expression.



THINK BEFORE YOU CODE

- Can you chain multiple filters together on a single expression? If so, in what order are they executed?
- How do you pass arguments to a filter (such as specific date formats)?

DEVELOPER NOTES / FILTER SYNTAX FORMULATION

```
// Output Transformation Strategy
```

Expression	Pipe Filter	Example Output

eventName	uppercase	"TECHFEST 2026"
participantCount	number	"1,500"
eventDate	date:'medium'	"Oct 24, 2026"
registrationFee	currency	"\$50.00"

TASK 6: PARTICIPANT REGISTRATION FORM FOUNDATION

Requirement: Construct the HTML form for new participant registrations. This is the core data entry point for the portal.

Required Fields & Constraints:

- Participant Name (Required, Minimum length rules, Maximum length rules)
- Email Address (Required, Must be a valid email format)



TECHNICAL HINT

Utilize standard HTML5 attributes like `required` and `type="email"` alongside AngularJS directives such as `ng-minlength` and `ng-maxlength`.
Use `ng-show` to display red error messages *only* when a validation rule is violated.



THINK BEFORE YOU CODE

- Why is it important to assign a name attribute to both the `<form>` and its `<input>` elements when working with AngularJS validation?
- Should error messages be visible immediately when the page loads, or only after the user interacts with the input?

DEVELOPER NOTES / FORM DOM STRUCTURE

```
// Form Architecture Requirements
```

```
FORM NAME: "registrationForm" (Requires 'novalidate' attribute)
```

```
FIELD 1: Participant Name
```

```
- Directive: ng-model="newParticipant.name"
```

```
- Validation: required, ng-minlength="3"
```

```
FIELD 2: Email Address
```

```
- Directive: ng-model="newParticipant.email"
```

```
- Type: "email", Validation: required
```

TASK 7: FORM STATE MONITORING

Requirement: Expose the internal state of the form to the UI to aid in debugging and user experience. Display the boolean values of the following form properties dynamically on the screen:

- `$formName.$valid`
- `$formName.$invalid`
- `$formName.$dirty`
- `$formName.$pristine`
- `$formName.fieldName.$touched`
- `$formName.fieldName.$valid`
- `$formName.fieldName.$invalid`



TECHNICAL HINT

Bind these properties within `{{ }}` expressions next to your form. Observe how these boolean values automatically toggle from `true` to `false` as you type into the inputs and blur focus.



THINK BEFORE YOU CODE

- What is the specific difference between a field being `$dirty` and a field being `$touched`?
- How can you use these state properties to prevent premature error messages from displaying?

DEVELOPER NOTES / STATE PROPERTY OBSERVATIONS

```
// State Tracking Matrix
```

STATE VARIABLE	TRIGGER CONDITION
----------------	-------------------

-------	--

form.\$pristine	User has not interacted with any field yet
-----------------	--

form.\$dirty	User has typed in at least one field
--------------	--------------------------------------

form.emailField.\$touched	User clicked the email field and then blurred
---------------------------	---

form.emailField.\$invalid	User typed an improperly formatted email
---------------------------	--

form.\$valid	ALL fields pass their respective validations
--------------	--

TASK 8 & 9: FORM SUBMISSION & DATA STORAGE

Requirement: Intercept the form submission process and store the newly registered participant's data persistently within the current session.

- Ensure the form does not perform a traditional page reload.
- Extract the data from the form's scope model.
- Append this new participant object to the array you initialized in Task 2.
- Clear the form inputs after successful storage.



TECHNICAL HINT

Use the `ng-submit` directive on the `<form>` tag to call a specific function inside your controller.

Inside the controller function, construct a new object and use the JavaScript array `push()` method.

THINK BEFORE YOU CODE

- Why is `ng-submit` on the form tag preferred over `ng-click` on the submit button?
- How do you reset the form state back to `$pristine` after pushing the data?

DEVELOPER NOTES / CONTROLLER FUNCTION LOGIC

```
// Controller Action: Form Submission  
  
FUNCTION submitRegistration() {  
  1. Verify form state (Check if $valid is true)  
  2. Construct new participant object from ng-model variables  
  3. Push new object into the existing 'participants' array  
  4. Reset ng-model variables to clear the inputs  
  5. Call $setPristine() and $setUntouched() on the form object  
}
```

TASK 10: DYNAMIC TABLE REPRESENTATION

Requirement: Construct a responsive data table to display all registered participants, including both pre-existing data and newly submitted data.

Required Columns:

- Participant Name
- Email



TECHNICAL HINT

Use ng-repeat on the <tr> (table row) element inside the <tbody>. Because of Two-way Data Binding, this table will update automatically the moment a new participant is pushed to the array in Task 9.



THINK BEFORE YOU CODE

- Do you need to write custom DOM manipulation code to update this table when a new user registers? Why or why not?
- How would you implement an \$index variable to show row numbers?

DEVELOPER NOTES / TABLE DOM STRUCTURE

```
// Dynamic Table Structure
```

```
TABLE HEADERS:
```

- Participant Name
- Email Address

```
TABLE BODY:
```

- Row Element (<tr>) bound with ng-repeat
- Data Cell (<td>) binding -> item.name
- Data Cell (<td>) binding -> item.email

```
// Note: Two-way binding ensures this updates automatically!
```

ADVANCED REQUIREMENT

BONUS CHALLENGE: REGEX VALIDATION

Requirement: Add a "Phone Number" field to the registration form. Implement strict validation so the system only accepts exactly 10 numeric digits. No letters, no spaces, no special characters.



TECHNICAL HINT

Utilize the `ng-pattern` directive with a Regular Expression to enforce the numeric format constraint.



THINK BEFORE YOU CODE

- What is the standard Regex pattern to match exactly 10 digits from start to finish?

DEVELOPER NOTES / REGEX DRAFTING

// Regex Pattern Formulation	
Start anchor symbol:	<code>^</code>
Character class (digits):	<code>[0-9]</code> (or <code>\d</code>)
Exact quantifier (10 times):	<code>{10}</code>
End anchor symbol:	<code>\$</code>
Combined ng-pattern regex string:	<code>"^[0-9]{10}\$"</code>

CHALLENGE YOURSELF: UI ENHANCEMENTS

Take your application to a professional standard by implementing the following dynamic UI directives:

✓ **ng-class**

Highlight the participant count dynamically. Apply a CSS class that turns the text **Green** if Count ≥ 100 , and **Red** if Count < 100 .

✓ **ng-style**

Allow the user to change the Organizer Name text color dynamically using an input field and inline styles.

✓ **ng-disabled**

Disable the "Register" submit button completely until the entire form passes all validation rules (`$valid`).

✓ **ng-readonly**

If you implement an Event ID field, ensure it is set to `readonly` so organizers cannot accidentally overwrite system IDs.

✓ **ng-init**

Use this directive to initialize a default Event Code immediately upon view load.

SELF-ASSESSMENT: STUDENT CHECKLIST

Before submitting your bootcamp project for evaluation, verify that you have successfully implemented and tested the following features:

- ☐ AngularJS Module Created successfully
- ☐ Application Controller Created and registered
- ☐ Data models bound using Scope Variables
- ☐ Core Dashboard Layout Completed
- ☐ `ng-model` utilized for two-way binding
- ☐ `ng-click` implemented for user interactions
- ☐ `ng-repeat` rendering lists and tables
- ☐ `ng-options` powering the dropdown menu
- ☐ `ng-switch` handling conditional UI states
- ☐ `ng-show` conditionally displaying error messages
- ☐ `ng-class` applying dynamic CSS styling
- ☐ `ng-style` updating inline styles
- ☐ `ng-disabled` securing the submit button

- ☐ `ng-readonly` locking specific inputs
-
- ☐ Pipe Filters (uppercase, date, etc.) actively formatting data
-
- ☐ HTML Registration Form Created
-
- ☐ Form State Validation Working correctly
-
- ☐ `ng-submit` capturing form payloads without reload
-
- ☐ Form Data Stored to the internal array structure
-
- ☐ Dynamic Table Displaying all stored participant records
-

PROJECT EVALUATION RUBRIC

Your portal will be assessed based on technical implementation, code quality, and adherence to requirements.

CRITERIA	DESCRIPTION	MARKS
Dashboard Implementation	Proper setup of initial variables and two-way binding mechanisms.	10
AngularJS Directives	Effective use of ng-repeat, ng-options, ng-switch, ng-class, etc.	20
Data Formatting (Filters)	Correct application of pipe filters in expressions.	10
Form Validation	Implementation of required, minlength, pattern, and error messages.	20
Event Handling (ng-submit)	Capturing form data, array manipulation, and preventing default refresh.	15
Dynamic Table Binding	Real-time updates of the participant table without manual DOM code.	15
UI/UX Design	Professional corporate styling, responsiveness, and clear layout.	5
Code Quality		5
TOTAL MAXIMUM SCORE		100

CRITERIA	DESCRIPTION	MARKS
	Clean, readable code, correct indentation, and semantic markup.	
TOTAL MAXIMUM SCORE		100

Evaluator Signature: _____

Date: _____